Timothy Song

Professor Shim

DATA 260

2 November 2026

Homework 8 Report

GitHub Repository: https://github.com/TimothyYSong/data260-9981

Section 0 - Personal Configuration and Domain

Hardware: MacBook Pro M4 Pro, 14 cores, 24 GB memory
Model:
Tagged commit hash:
SID4: 9981
PORT_BASE: 8081
PREFIX: s9981
SEED: 9981
VERIFY_SEED: 269981
DOMAIN_ID: 5 (Local restaurant inspections)

Part 1 - Backend Service Request/Response over Kafka

Required Screenshots and Evidence

1.

```
(base) optim@Timothys-MacBook-Pro data260-9981 % docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED          STATUS          PORTS                     NAMES
d349275cc63d   data260-9981-worker   "python kafka_inspec…"   7 minutes ago    Up 7 minutes    8081/tcp                  s9981-worker
8c308d4ebe4f   data260-9981-api      "uvicorn hw08_api:ap…"   10 minutes ago   Up 10 minutes   127.0.0.1:8081->8081/tcp  s9981-api
b0d73a44ea3e   apache/kafka:3.9.1    "/__cacert_entrypoin…"   16 minutes ago   Up 16 minutes   127.0.0.1:9092->9092/tcp  s9981-kafka
(base) optim@Timothys-MacBook-Pro data260-9981 % docker compose logs worker | grep "Consumer is ready and listening"
s9981-worker  | 2026-11-02 03:51:26,874 INFO Consumer is ready and listening...
(base) optim@Timothys-MacBook-Pro data260-9981 % docker compose logs api | grep -E "Application startup complete|Uvicorn running"
s9981-api  | INFO:     Application startup complete.
s9981-api  | INFO:     Uvicorn running on http://0.0.0.0:8081 (Press CTRL+C to quit)
(base) optim@Timothys-MacBook-Pro data260-9981 %
```

Figure 1. Docker Services Running: Kafka, the inspection worker, and FastAPI are running in separate Docker containers. The worker confirms that it is listening for requests, and FastAPI is accessible at 127.0.0.1:8081.

2.

```
(base) optim@Timothys-MacBook-Pro data260-9981 %
docker compose run --rm --no-deps \
  -v "$PWD/code:/app" \
  worker python homework_demo.py

Container data260-9981-worker-run-cd2633fca8b7 Creating
Container data260-9981-worker-run-cd2633fca8b7 Created
2026-11-02 04:04:51,717 INFO === CREATE_INSPECTION ===
2026-11-02 04:04:51,717 INFO CREATE correlation_id: e3885b76-49a5-4084-aa5c-8ebf712e08c
4
2026-11-02 04:04:51,717 INFO CLIENT SENT REQUEST: {"correlation_id": "e3885b76-49a5-408
4-aa5c-8ebf712e08c4", "reply_to": "s9981_entity_responses", "operation": "CREATE_INSPEC
TION", "data": {"restaurant_name": "O2 Valley", "inspection_type": "routine", "inspecti
on_date": "2026-11-01T12:00:00", "description": "HW8 complete Kafka demonstration"}}
2026-11-02 04:04:51,736 INFO CLIENT RECEIVED RESPONSE: {"success": true, "inspection_id
": "27a7e3e4-d6c4-4735-b0c8-30730bc83381", "message": "Entity created", "correlation_id
": "e3885b76-49a5-4084-aa5c-8ebf712e08c4"}
2026-11-02 04:04:51,736 INFO CREATE RESPONSE: {"success": true, "inspection_id": "27a7e
3e4-d6c4-4735-b0c8-30730bc83381", "message": "Entity created", "correlation_id": "e3885
b76-49a5-4084-aa5c-8ebf712e08c4"}
2026-11-02 04:04:51,736 INFO Generated inspection_id: 27a7e3e4-d6c4-4735-b0c8-30730bc83
381
2026-11-02 04:04:51,736 INFO CREATE correlation_id preserved: PASS
2026-11-02 04:04:51,736 INFO === GET_INSPECTION ===
2026-11-02 04:04:51,736 INFO GET correlation_id: d71c4cab-cd74-46ab-943e-23102b65f896
2026-11-02 04:04:51,736 INFO CLIENT SENT REQUEST: {"correlation_id": "d71c4cab-cd74-46a
b-943e-23102b65f896", "reply_to": "s9981_entity_responses", "operation": "GET_INSPECTIO
N", "data": {"inspection_id": "27a7e3e4-d6c4-4735-b0c8-30730bc83381"}}
2026-11-02 04:04:51,740 INFO CLIENT RECEIVED RESPONSE: {"success": true, "entity": {"in
spection_id": "27a7e3e4-d6c4-4735-b0c8-30730bc83381", "restaurant_name": "O2 Valley", "
description": "HW8 complete Kafka demonstration", "status": "pending", "inspection_type
": "routine", "inspection_date": "2026-11-01T12:00:00"}, "correlation_id": "d71c4cab-cd
74-46ab-943e-23102b65f896"}
2026-11-02 04:04:51,740 INFO GET RESPONSE: {"success": true, "entity": {"inspection_id"
: "27a7e3e4-d6c4-4735-b0c8-30730bc83381", "restaurant_name": "O2 Valley", "description"
: "HW8 complete Kafka demonstration", "status": "pending", "inspection_type": "routine"
, "inspection_date": "2026-11-01T12:00:00"}, "correlation_id": "d71c4cab-cd74-46ab-943e
-23102b65f896"}
2026-11-02 04:04:51,740 INFO RETRIEVED ENTITY: {"inspection_id": "27a7e3e4-d6c4-4735-b0
c8-30730bc83381", "restaurant_name": "O2 Valley", "description": "HW8 complete Kafka de
monstration", "status": "pending", "inspection_type": "routine", "inspection_date": "20
26-11-01T12:00:00"}
2026-11-02 04:04:51,740 INFO GET correlation_id preserved: PASS
2026-11-02 04:04:51,740 INFO CREATE -> GET round trip: PASS
(base) optim@Timothys-MacBook-Pro data260-9981 %
```

Figure 2. CREATE and GET Inspection Execution: The Kafka client successfully creates an inspection, generates a UUID, and retrieves the same inspection using its ID. Both operations return success: true and preserve their respective correlation IDs.

3.



Figure 3. Kafka Message Flow: The client sends CREATE and GET inspection requests through s9981_entity_requests. The worker receives and processes each request, publishes responses to s9981_entity_responses, and the client receives the corresponding responses with unchanged correlation IDs.

4.

The complete CREATE_INSPECTION → GET_INSPECTION message sequence is shown in Figure 3. Both operations follow the client send → worker receive → process → reply → client response flow, preserving their respective correlation IDs.

5.

kafka_inspection_client.py 1, U

code > kafka_inspection_client.py > ...

```python
PREFIX = "s9981"
REQUEST_TOPIC = f"{PREFIX}_entity_requests"
RESPONSE_TOPIC = f"{PREFIX}_entity_responses"
BOOTSTRAP_SERVERS = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092")

logger = logging.getLogger(__name__)


class InspectionKafkaClient:
    def __init__(self):
        self.producer = KafkaProducer(
            bootstrap_servers=BOOTSTRAP_SERVERS,
            value_serializer=lambda value: json.dumps(value).encode("utf-8"),
        )

        self.consumer = KafkaConsumer(
            RESPONSE_TOPIC,
            bootstrap_servers=BOOTSTRAP_SERVERS,
            group_id=None,
            auto_offset_reset="latest",
            enable_auto_commit=False,
            value_deserializer=lambda value: json.loads(value.decode("utf-8")),
        )

        self.consumer.poll(timeout_ms=1000)

    def request(self, operation, data, correlation_id=None, timeout=20):
        if correlation_id is None:
            correlation_id = str(uuid4())

        request = {
            "correlation_id": correlation_id,
            "reply_to": RESPONSE_TOPIC,
            "operation": operation,
            "data": data,
        }

        logger.info("CLIENT SENT REQUEST: %s", json.dumps(request))
        self.producer.send(REQUEST_TOPIC, value=request).get(timeout=10)

        deadline = time.monotonic() + timeout

        while time.monotonic() < deadline:
            records = self.consumer.poll(timeout_ms=500)

            for messages in records.values():
                for message in messages:
                    response = message.value

                    if response.get("correlation_id") == correlation_id:
                        logger.info(
                            "CLIENT RECEIVED RESPONSE: %s",
                            json.dumps(response),
                        )
                        return response

        raise TimeoutError(
            f"No matching Kafka response for correlation_id={correlation_id}"
        )
```

Figure 4. Kafka Client: The client publishes requests to s9981_entity_requests and consumes responses from s9981_entity_responses. Each request includes the operation, data, reply topic, and correlation ID, which is used to match the corresponding response.

homework_demo.py U

code > homework_demo.py > ...

```python
def main():
    client = InspectionKafkaClient()

    try:
        create_id = str(uuid4())

        logger.info("=== CREATE_INSPECTION ===")
        logger.info("CREATE correlation_id: %s", create_id)

        create_response = client.request(
            operation="CREATE_INSPECTION",
            data={
                "restaurant_name": "O2 Valley",
                "inspection_type": "routine",
                "inspection_date": "2026-11-01T12:00:00",
                "description": "HW8 complete Kafka demonstration",
            },
            correlation_id=create_id,
        )

        logger.info(
            "CREATE RESPONSE: %s",
            json.dumps(create_response),
        )

        assert create_response["success"] is True
        assert create_response["correlation_id"] == create_id

        inspection_id = create_response["inspection_id"]

        logger.info("Generated inspection_id: %s", inspection_id)
        logger.info("CREATE correlation_id preserved: PASS")

        get_id = str(uuid4())

        logger.info("=== GET_INSPECTION ===")
        logger.info("GET correlation_id: %s", get_id)

        get_response = client.request(
            operation="GET_INSPECTION",
            data={"inspection_id": inspection_id},
            correlation_id=get_id,
        )

        logger.info(
            "GET RESPONSE: %s",
            json.dumps(get_response),
        )

        assert get_response["success"] is True
        assert get_response["correlation_id"] == get_id
        assert get_response["entity"]["inspection_id"] == inspection_id

        logger.info(
            "RETRIEVED ENTITY: %s",
            json.dumps(get_response["entity"]),
        )

        logger.info("GET correlation_id preserved: PASS")
        logger.info("CREATE -> GET round trip: PASS")

    finally:
        client.close()
```

Figure 5. Kafka Demonstration: The homework_demo.py script sends a CREATE_INSPECTION request, extracts the generated inspection ID, and uses it to retrieve the same inspection through GET_INSPECTION. Both operations verify successful responses and preserved correlation IDs.

kafka_inspection_worker.py 1, U

code > kafka_inspection_worker.py > ...

```python
PREFIX = "s9981"
REQUEST_TOPIC = f"{PREFIX}_entity_requests"
RESPONSE_TOPIC = f"{PREFIX}_entity_responses"
BOOTSTRAP_SERVERS = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092")

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
)
logger = logging.getLogger(__name__)

inspections = {}


def process_request(request):
    operation = request.get("operation")
    data = request.get("data")

    if operation not in ("CREATE_INSPECTION", "GET_INSPECTION"):
        return {"success": False, "error": "Unknown operation"}

    if not isinstance(data, dict):
        return {"success": False, "error": "Invalid data object"}

    if operation == "CREATE_INSPECTION":
        try:
            inspection = InspectionCreate.model_validate(data)

            if not inspection.restaurant_name.strip():
                return {
                    "success": False,
                    "error": "restaurant_name cannot be empty",
                }

            inspection_id = str(uuid4())
            inspections[inspection_id] = {
                "inspection_id": inspection_id,
                **inspection.model_dump(mode="json"),
            }

            return {
                "success": True,
                "inspection_id": inspection_id,
                "message": "Entity created",
            }

        except ValidationError as exc:
            return {
                "success": False,
                "error": f"Invalid inspection data: {exc.error_count()} validation error(s)",
            }

    if operation == "GET_INSPECTION":
        inspection_id = data.get("inspection_id")

        if not isinstance(inspection_id, str) or not inspection_id.strip():
            return {"success": False, "error": "Invalid inspection_id"}

        entity = inspections.get(inspection_id)

        if entity is None:
            return {"success": False, "error": "Entity not found"}

        return {"success": True, "entity": entity}

    return {"success": False, "error": "Unknown operation"}
```

kafka_inspection_worker.py 1, U

code > kafka_inspection_worker.py > ...

```python
def main():
    producer = KafkaProducer(
        bootstrap_servers=BOOTSTRAP_SERVERS,
        value_serializer=lambda value: json.dumps(value).encode("utf-8"),
    )

    consumer = KafkaConsumer(
        REQUEST_TOPIC,
        bootstrap_servers=BOOTSTRAP_SERVERS,
        group_id=f"{PREFIX}_inspection_workers",
        auto_offset_reset="earliest",
        enable_auto_commit=False,
        value_deserializer=lambda value: json.loads(value.decode("utf-8")),
    )

    logger.info("Consumer is ready and listening...")

    for message in consumer:
        request = message.value
        logger.info("WORKER RECEIVED: %s", json.dumps(request))

        correlation_id = request.get("correlation_id")
        reply_to = request.get("reply_to")

        if not isinstance(reply_to, str) or reply_to != RESPONSE_TOPIC:
            logger.error("Invalid reply_to; request skipped")
            consumer.commit()
            continue

        try:
            response = process_request(request)
            response["correlation_id"] = correlation_id

            logger.info("WORKER PROCESSED: %s", request.get("operation"))

            producer.send(reply_to, value=response).get(timeout=15)
            logger.info("WORKER SENT RESPONSE: %s", json.dumps(response))

            consumer.commit()

        except Exception:
            logger.exception("Worker processing failed; offset not committed")
            raise
```

Figure 6. Kafka Worker: The worker validates CREATE_INSPECTION and GET_INSPECTION requests, stores inspections in an in-memory dictionary, and generates UUIDs. It consumes Kafka requests, processes each operation, and publishes responses with unchanged correlation IDs.



Figure 7. FastAPI Wrapper: The API exposes CREATE_INSPECTION and GET_INSPECTION endpoints, forwards requests through the Kafka client, and returns the worker responses while preserving correlation IDs.

Kafka Topics, Message Schemas, and Correlation ID Flow

Topics: s9981_entity_requests receives client requests, and s9981_entity_responses carries worker responses.

Request schema:

```JSON
{
  "correlation_id": "unique-uuid",
  "reply_to": "s9981_entity_responses",
  "operation": "CREATE_INSPECTION",
  "data": {
    "restaurant_name": "O2 Valley",
    "inspection_type": "routine",
    "inspection_date": "2026-11-01T12:00:00"
  }
}
```

Response schema:

```JSON
{
  "success": true,
  "inspection_id": "generated-uuid",
  "message": "Entity created",
  "correlation_id": "same-request-uuid"
}
```

Correlation ID flow: The client generates a correlation ID and includes it in the Kafka request. The worker reads that ID, processes the operation, and includes the unchanged ID in its response. The client matches the response using the same ID. Figure 3 verifies this flow for both CREATE_INSPECTION and GET_INSPECTION.

Part 2 - Three-Agent Kafka Pipeline

Submit:

Your Planner, Writer, and Reviewer code.

planner.py 1, U

code > planner.py > ...

```python
PREFIX = "s9981"
INPUT_TOPIC = f"{PREFIX}_inbox"
OUTPUT_TOPIC = f"{PREFIX}_tasks"
BOOTSTRAP_SERVERS = os.getenv(
    "KAFKA_BOOTSTRAP_SERVERS", "localhost:9092"
)

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
)
logging.getLogger("kafka").setLevel(logging.WARNING)
logger = logging.getLogger(__name__)


def main():
    producer = KafkaProducer(
        bootstrap_servers=BOOTSTRAP_SERVERS,
        value_serializer=lambda value: json.dumps(value).encode("utf-8"),
    )

    consumer = KafkaConsumer(
        INPUT_TOPIC,
        bootstrap_servers=BOOTSTRAP_SERVERS,
        group_id=f"{PREFIX}_planners",
        auto_offset_reset="earliest",
        enable_auto_commit=False,
        value_deserializer=lambda value: json.loads(value.decode("utf-8")),
    )

    logger.info("PLANNER READY: %s -> %s", INPUT_TOPIC, OUTPUT_TOPIC)

    for message in consumer:
        request = message.value
        logger.info("PLANNER RECEIVED: %s", json.dumps(request))

        correlation_id = request["correlation_id"]
        question = request["question"]

        plan = {
            "correlation_id": correlation_id,
            "question": question,
            "steps": [
                "Identify the restaurant inspection objective.",
                "List relevant food safety inspection checks.",
                "Organize the checks into a clear answer.",
            ],
        }

        producer.send(OUTPUT_TOPIC, value=plan).get(timeout=15)
        logger.info("PLANNER SENT: %s", json.dumps(plan))

        consumer.commit()
```

Figure 8. Planner Agent: The Planner consumes questions from s9981_inbox, generates a structured plan, and publishes it to s9981_tasks while preserving the correlation ID.

writer.py 1, U

code > writer.py > ...

```python
PREFIX = "s9981"
INPUT_TOPIC = f"{PREFIX}_tasks"
OUTPUT_TOPIC = f"{PREFIX}_drafts"
BOOTSTRAP_SERVERS = os.getenv(
    "KAFKA_BOOTSTRAP_SERVERS", "localhost:9092"
)

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
)
logging.getLogger("kafka").setLevel(logging.WARNING)
logger = logging.getLogger(__name__)


def main():
    producer = KafkaProducer(
        bootstrap_servers=BOOTSTRAP_SERVERS,
        value_serializer=lambda value: json.dumps(value).encode("utf-8"),
    )

    consumer = KafkaConsumer(
        INPUT_TOPIC,
        bootstrap_servers=BOOTSTRAP_SERVERS,
        group_id=f"{PREFIX}_writers",
        auto_offset_reset="earliest",
        enable_auto_commit=False,
        value_deserializer=lambda value: json.loads(value.decode("utf-8")),
    )

    logger.info("WRITER READY: %s -> %s", INPUT_TOPIC, OUTPUT_TOPIC)

    for message in consumer:
        task = message.value
        logger.info("WRITER RECEIVED: %s", json.dumps(task))

        correlation_id = task["correlation_id"]
        question = task["question"]
        steps = task["steps"]

        answer = (
            f"Question: {question} "
            "A restaurant inspection checklist should cover food "
            "storage temperatures, employee hygiene, sanitation, "
            "cross-contamination prevention, pest control, and "
            "equipment cleanliness."
        )

        draft = {
            "correlation_id": correlation_id,
            "question": question,
            "plan": steps,
            "draft": answer,
        }

        producer.send(OUTPUT_TOPIC, value=draft).get(timeout=15)
        logger.info("WRITER SENT: %s", json.dumps(draft))

        consumer.commit()
```

Figure 9. Writer Agent: The Writer consumes plans from s9981_tasks, generates draft answers, and publishes them to s9981_drafts while preserving the correlation ID.

reviewer.py 1, U

code > reviewer.py > ...

```python
PREFIX = "s9981"
INPUT_TOPIC = f"{PREFIX}_drafts"
OUTPUT_TOPIC = f"{PREFIX}_final"
BOOTSTRAP_SERVERS = os.getenv(
    "KAFKA_BOOTSTRAP_SERVERS", "localhost:9092"
)

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
)
logging.getLogger("kafka").setLevel(logging.WARNING)
logger = logging.getLogger(__name__)


def main():
    producer = KafkaProducer(
        bootstrap_servers=BOOTSTRAP_SERVERS,
        value_serializer=lambda value: json.dumps(value).encode("utf-8"),
    )

    consumer = KafkaConsumer(
        INPUT_TOPIC,
        bootstrap_servers=BOOTSTRAP_SERVERS,
        group_id=f"{PREFIX}_reviewers",
        auto_offset_reset="earliest",
        enable_auto_commit=False,
        value_deserializer=lambda value: json.loads(value.decode("utf-8")),
    )

    logger.info("REVIEWER READY: %s -> %s", INPUT_TOPIC, OUTPUT_TOPIC)

    for message in consumer:
        draft_message = message.value
        logger.info("REVIEWER RECEIVED: %s", json.dumps(draft_message))

        correlation_id = draft_message["correlation_id"]
        draft = draft_message["draft"]

        if isinstance(draft, str) and draft.strip():
            status = "approved"
            answer = draft
        else:
            status = "rejected"
            answer = "The draft was empty or invalid."

        final_message = {
            "correlation_id": correlation_id,
            "status": status,
            "answer": answer,
        }

        producer.send(OUTPUT_TOPIC, value=final_message).get(timeout=15)
        logger.info("REVIEWER SENT: %s", json.dumps(final_message))

        consumer.commit()
```

Figure 10. Reviewer Agent: The Reviewer consumes drafts from s9981_drafts, validates the draft content, and publishes the final JSON response to s9981_final with the original correlation ID.

One complete message journey from _inbox to _final, shown as log lines for each stage.

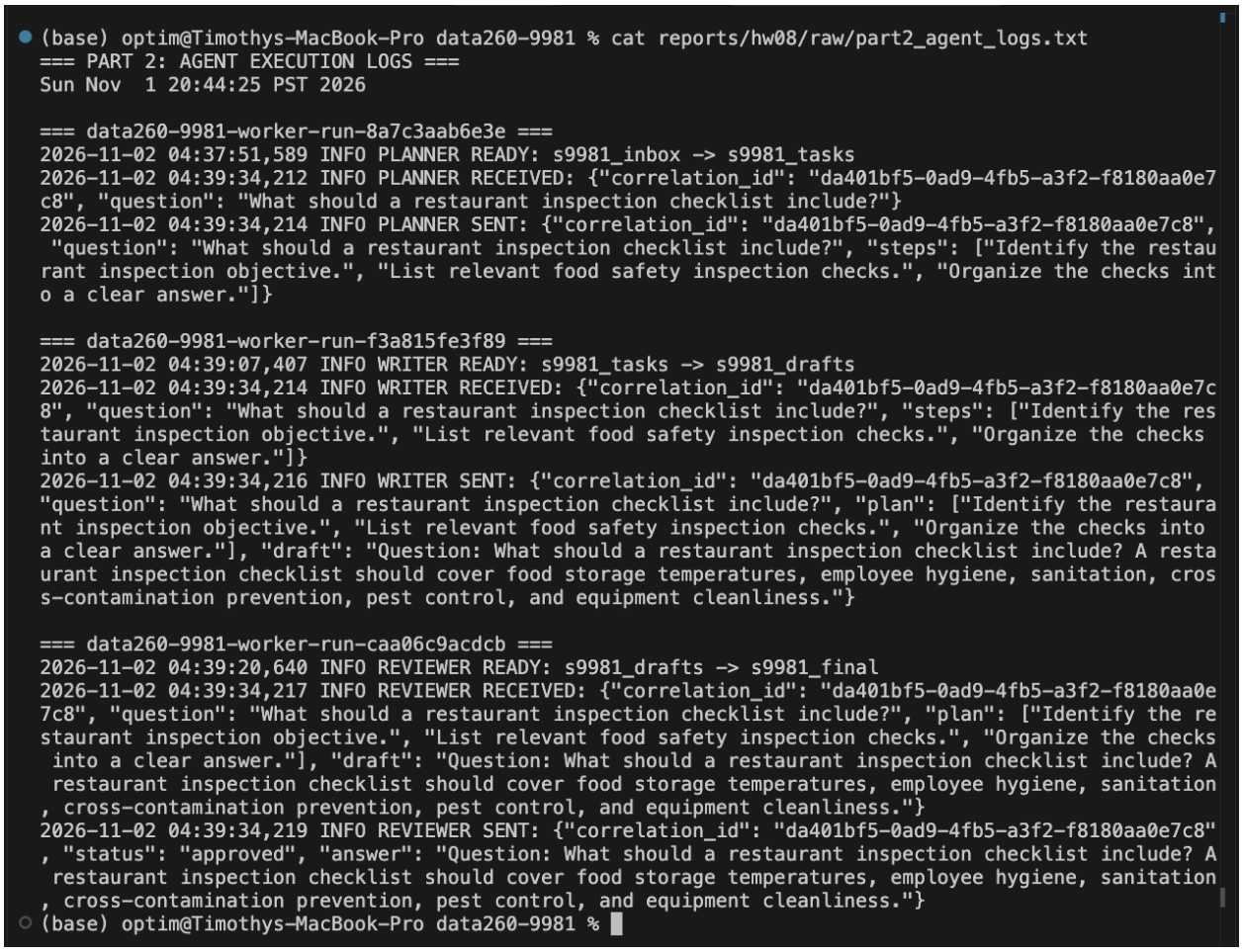


Figure 11. Three-Agent Kafka Message Journey: The Planner consumes the question from s9981_inbox and publishes a plan to s9981_tasks. The Writer generates a draft in s9981_drafts, and the Reviewer publishes the approved answer to s9981_final. All stages preserve the original correlation ID.

The final JSON message with its status and answer.

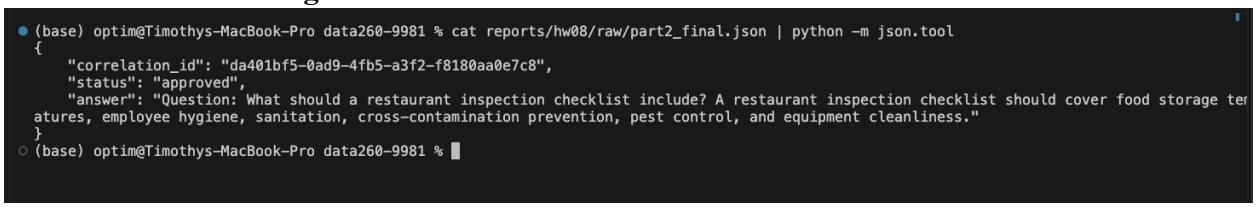


Figure 12. Final Kafka JSON Response: The final message contains an approved status, the completed restaurant inspection answer, and the original correlation ID preserved throughout the three-agent pipeline.

A short report explaining how Kafka helped your agents coordinate.

Kafka allows the Planner, Writer, and Reviewer to operate as independent processes without directly calling one another. Each agent consumes messages from its assigned topic and publishes its output to the next topic. The correlation ID remains unchanged across all four topics, allowing the complete message journey to be traced from the original question to the approved answer.

Part 3 - Failure Behavior

Submit

The completed failure table above, with a matching log excerpt for every row.

Failure scenario	Before fix	After fix
Duplicate delivery	Replaying the same correlation ID published 2 drafts.	Idempotency skipped the duplicate; only 1 draft published.
Consumer killed mid-work	Writer killed before committing offset 3; lag remained 1.	Restarted Writer processed the task; committed offset advanced to 4, lag 0.
Poison message	Missing steps caused KeyError and crashed Writer.	3 attempts, then sent to s9981_dead_letter; next valid message processed.
Two partitions/two consumers	20 related tasks split evenly across 2 partitions and Writers.	Entity key routed all 20 to partition 1; processed in order from 1–20.



Figure 13. Duplicate Delivery Before and After Idempotency: Before the fix, replaying the same correlation ID caused the Writer to publish two drafts. After the fix, the Writer published one draft and skipped the duplicate, preventing an additional side effect.

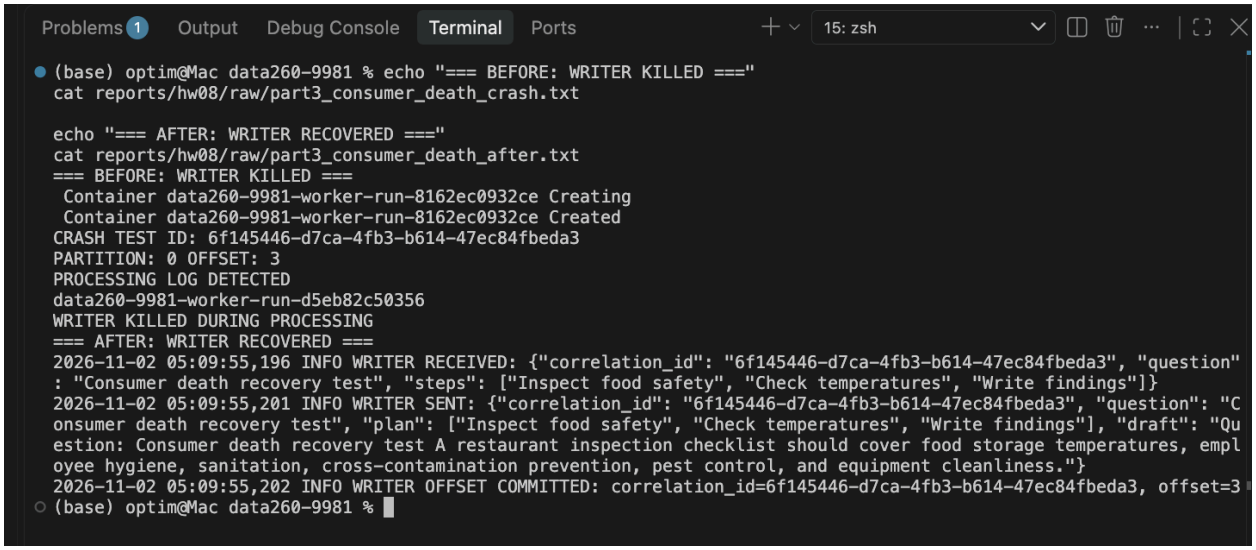


Figure 14. Consumer Death and Recovery: The Writer was terminated during processing, leaving the task uncommitted at offset 3 with lag 1. After restarting, Kafka redelivered the task, and the Writer successfully published its draft. The committed offset advanced to 4 with lag 0.

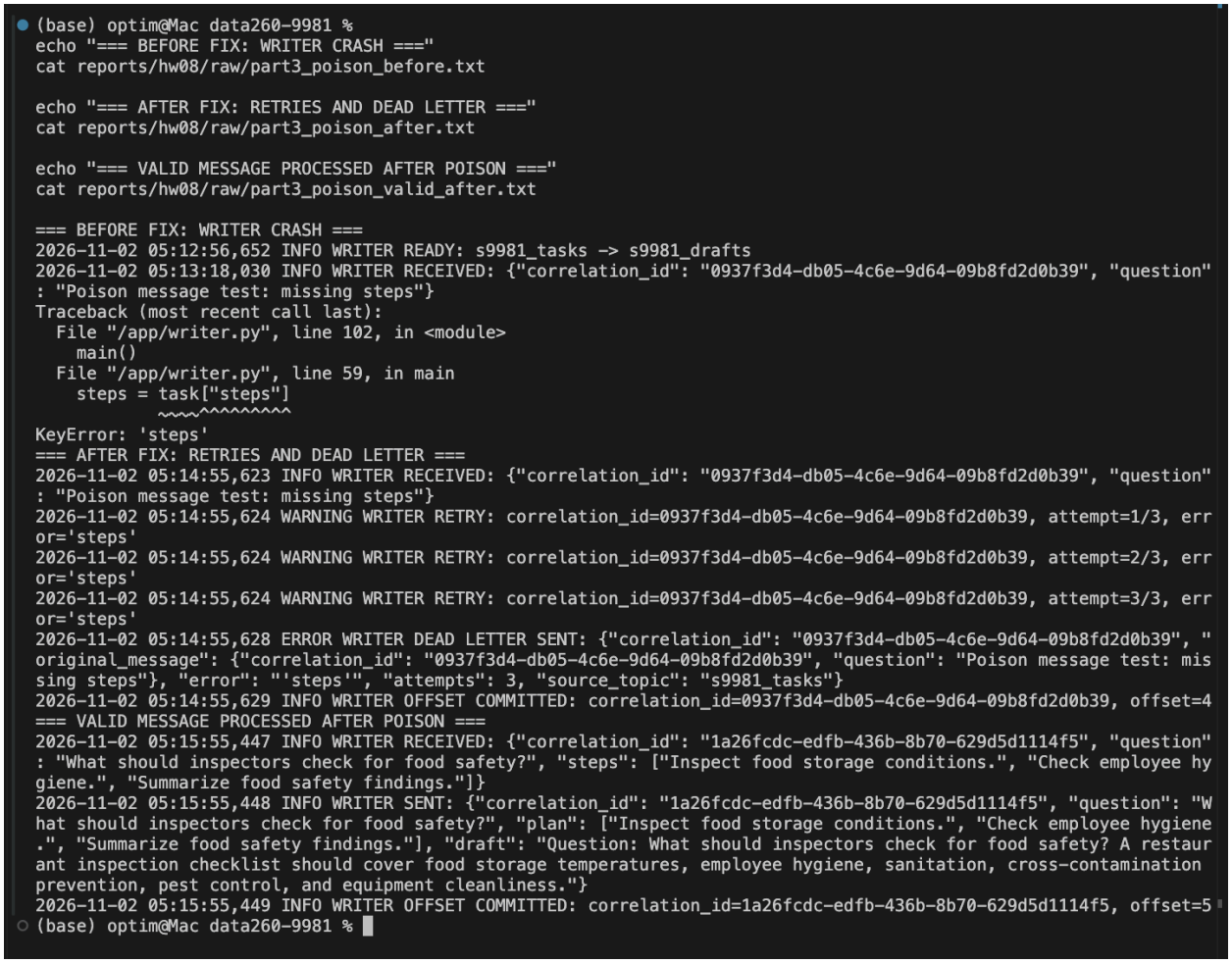


Figure 15. Poison Message Before and After Recovery: Before the fix, a message missing the required steps field crashed the Writer with a KeyError. After the fix, the Writer attempted processing three times, sent the failed message to s9981_dead_letter, and successfully processed the next valid message.

```
(base) optim@Mac data260-9981 %
echo "=== BEFORE FIX: TWO PARTITIONS ==="
cat reports/hw08/raw/part3_ordering_before_logs.txt

echo "=== AFTER FIX: ENTITY-BASED KEY ==="
cat reports/hw08/raw/part3_ordering_after_clean.txt

=== BEFORE FIX: TWO PARTITIONS ===
=== WRITER 1 ===
2026-11-02 05:22:03,339 INFO WRITER ORDER: sequence=2, partition=0, offset=6, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,345 INFO WRITER ORDER: sequence=3, partition=0, offset=7, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,347 INFO WRITER ORDER: sequence=4, partition=0, offset=8, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,350 INFO WRITER ORDER: sequence=7, partition=0, offset=9, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,353 INFO WRITER ORDER: sequence=9, partition=0, offset=10, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,356 INFO WRITER ORDER: sequence=14, partition=0, offset=11, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,358 INFO WRITER ORDER: sequence=16, partition=0, offset=12, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,362 INFO WRITER ORDER: sequence=17, partition=0, offset=13, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,365 INFO WRITER ORDER: sequence=18, partition=0, offset=14, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,367 INFO WRITER ORDER: sequence=20, partition=0, offset=15, entity_id=restaurant-9981, pid=1
=== WRITER 2 ===
2026-11-02 05:22:03,338 INFO WRITER ORDER: sequence=1, partition=1, offset=3, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,342 INFO WRITER ORDER: sequence=5, partition=1, offset=4, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,346 INFO WRITER ORDER: sequence=6, partition=1, offset=5, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,347 INFO WRITER ORDER: sequence=8, partition=1, offset=6, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,351 INFO WRITER ORDER: sequence=10, partition=1, offset=7, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,353 INFO WRITER ORDER: sequence=11, partition=1, offset=8, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,356 INFO WRITER ORDER: sequence=12, partition=1, offset=9, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,358 INFO WRITER ORDER: sequence=13, partition=1, offset=10, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,362 INFO WRITER ORDER: sequence=15, partition=1, offset=11, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:03,364 INFO WRITER ORDER: sequence=19, partition=1, offset=12, entity_id=restaurant-9981, pid=1
=== AFTER FIX: ENTITY-BASED KEY ===
2026-11-02 05:22:43,634 INFO WRITER ORDER: sequence=1, partition=1, offset=13, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,638 INFO WRITER ORDER: sequence=2, partition=1, offset=14, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,640 INFO WRITER ORDER: sequence=3, partition=1, offset=15, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,643 INFO WRITER ORDER: sequence=4, partition=1, offset=16, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,645 INFO WRITER ORDER: sequence=5, partition=1, offset=17, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,647 INFO WRITER ORDER: sequence=6, partition=1, offset=18, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,648 INFO WRITER ORDER: sequence=7, partition=1, offset=19, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,650 INFO WRITER ORDER: sequence=8, partition=1, offset=20, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,651 INFO WRITER ORDER: sequence=9, partition=1, offset=21, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,654 INFO WRITER ORDER: sequence=10, partition=1, offset=22, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,656 INFO WRITER ORDER: sequence=11, partition=1, offset=23, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,658 INFO WRITER ORDER: sequence=12, partition=1, offset=24, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,660 INFO WRITER ORDER: sequence=13, partition=1, offset=25, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,662 INFO WRITER ORDER: sequence=14, partition=1, offset=26, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,664 INFO WRITER ORDER: sequence=15, partition=1, offset=27, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,665 INFO WRITER ORDER: sequence=16, partition=1, offset=28, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,666 INFO WRITER ORDER: sequence=17, partition=1, offset=29, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,668 INFO WRITER ORDER: sequence=18, partition=1, offset=30, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,669 INFO WRITER ORDER: sequence=19, partition=1, offset=31, entity_id=restaurant-9981, pid=1
2026-11-02 05:22:43,670 INFO WRITER ORDER: sequence=20, partition=1, offset=32, entity_id=restaurant-9981, pid=1
(base) optim@Mac data260-9981 %
```

Figure 16. Kafka Ordering Before and After Entity-Based Partitioning: Before the fix, 20 related restaurant-inspection messages were distributed across two partitions and processed by two Writer instances, without a global ordering guarantee. After applying the restaurant ID as the partition key, all 20 messages were assigned to partition 1 and processed sequentially from 1 to 20 by Writer 2.

Your idempotency, offset-commit, retry/dead-letter, and partition-key code.

```
processed_ids = set()
```

```
if correlation_id in processed_ids:
    logger.info(
        "WRITER DUPLICATE SKIPPED: correlation_id=%s",
        correlation_id,
    )
    consumer.commit()
    continue
```

```
processed_ids.add(correlation_id)
```

Figure 17. Idempotency Implementation (code/writer.py): Tracks processed correlation IDs and skips duplicate messages to prevent repeated draft publication within the same Writer process.

```
consumer = KafkaConsumer(
    INPUT_TOPIC,
    bootstrap_servers=BOOTSTRAP_SERVERS,
    group_id=f"{PREFIX}_writers",
    auto_offset_reset="earliest",
    enable_auto_commit=False,
    value_deserializer=lambda value: json.loads(
        value.decode("utf-8")
    ),
)
```

```
producer.send(
    OUTPUT_TOPIC,
    value=draft,
).get(timeout=15)
```

```
consumer.commit()

logger.info(
    "WRITER OFFSET COMMITTED: "
    "correlation_id=%s, offset=%s",
    correlation_id,
    message.offset,
)
```

Figure 18. Manual Offset Commit (code/writer.py): Disables automatic offset commits and commits after successful draft publication or dead-letter delivery.

```
MAX_ATTEMPTS = 3
```

```
for attempt in range(1, MAX_ATTEMPTS + 1):
    try:
        draft = create_draft(task)

        producer.send(
            OUTPUT_TOPIC,
            value=draft,
        ).get(timeout=15)

        logger.info(
            "WRITER SENT: %s",
            json.dumps(draft),
        )

        processed_ids.add(correlation_id)
        break

    except (KeyError, TypeError, ValueError) as error:
        logger.warning(
            "WRITER RETRY: correlation_id=%s, "
            "attempt=%s/%s, error=%s",
            correlation_id,
            attempt,
            MAX_ATTEMPTS,
            error,
        )

        if attempt == MAX_ATTEMPTS:
            dead_letter = {
                "correlation_id": correlation_id,
                "original_message": task,
                "error": str(error),
                "attempts": MAX_ATTEMPTS,
                "source_topic": INPUT_TOPIC,
            }

            producer.send(
                DEAD_LETTER_TOPIC,
                value=dead_letter,
            ).get(timeout=15)

            logger.error(
                "WRITER DEAD LETTER SENT: %s",
                json.dumps(dead_letter),
            )
```

Figure 19. Retry and Dead-Letter Handling (code/writer.py): Retries failed messages up to three times before publishing them to s9981_dead_letter.



Figure 20. Entity-Based Partition Key (code/ordering_test.py): Compares explicit two-partition distribution with restaurant-ID-based partitioning to preserve the processing order of related messages.

Your matching logs must show the duplicate replay result, the consumer restart and offset behavior, the retry and _dead_letter result, and both ordering runs.

The following logs document the four failure experiments:

Experiment	Evidence	Raw log files
Duplicate delivery	Figure 13 — Duplicate replay before and after idempotency	part3_duplicate_before.txt, part3_duplicate_after.txt
Consumer death	Figure 14 — Writer termination, recovery, and offset commit	part3_consumer_death_crash.txt, part3_consumer_death_after.txt
Poison message	Figure 15 — Three retries, dead-letter publication, and subsequent valid message	part3_poison_before.txt, part3_poison_after.txt, part3_poison_valid_after.txt
Message ordering	Figure 16 — Two-partition baseline and entity-keyed ordering	part3_ordering_before_logs.txt, part3_ordering_after_clean.txt

All raw evidence files are located in reports/hw08/raw/. Final Kafka consumer-group verification showed zero lag on both partitions.

Your two-benefits/two-responsibilities comparison with the HW2 in-process graph.

In HW2, the agent graph executed sequentially through function calls within a single process. In HW8, Kafka enables asynchronous communication between the Planner, Writer, and Reviewer through message topics.

Two Benefits of Kafka

1. Independent scaling: Multiple Writer instances can process messages from different Kafka partitions concurrently, allowing individual components to scale independently.
2. Failure isolation and recovery: If a Writer crashes, Kafka retains uncommitted messages so they can be processed after the consumer restarts.

Two New Responsibilities

1. Duplicate delivery: Kafka messages may be processed more than once, requiring idempotency to prevent duplicate side effects.
2. Message ordering: Kafka guarantees ordering within a partition, not across partitions. Related messages must use a consistent entity-based partition key to preserve their sequence.

DATA 260 — Homework 8 AI Use Disclosure

1. What did you use an AI assistant for, and what did you do yourself?

I used ChatGPT to help interpret the HW8 instructions, plan implementation steps, draft and review Python/Kafka code, troubleshoot Docker and Kafka CLI commands, and organize the required report, logs, and results tables. This included guidance for the inspection request/response service, the Planner → Writer → Reviewer pipeline, and the four Part 3 failure experiments.

I ran the commands in my own development environment, edited and saved the source files, started and stopped the Kafka/worker containers, carried out the CREATE/GET and failure tests, checked the actual terminal output, and captured the report evidence. I checked results against the assignment handout and saved the real logs and CSV outputs under `reports/hw08/`.

2. What was one AI-produced output that was wrong or unsuitable, or one result you independently verified?

An AI-suggested Kafka consumer-group inspection command used `kafka-consumer-groups.sh` without its absolute path inside the Kafka container. That command did not run because the executable was not in the container's `PATH`.

3. How did you detect or verify the problem?

I executed the suggested command and observed the actual error: `exec: "kafka-consumer-groups.sh": executable file not found in $PATH`. I then ran the corrected command and inspected the consumer-group table rather than relying on the AI's predicted output. The table showed both `s9981_tasks` partitions caught up: partition 0 current/end offsets `16/16` and partition 1 `33/33`, with lag `0` on each.

```
## 4. What did you change, and why does it work now?

I used the executable's full path, `/opt/kafka/bin/kafka-consumer-groups.sh`, with the same bootstrap server and consumer group:

```bash
docker compose exec -T kafka \
 /opt/kafka/bin/kafka-consumer-groups.sh \
 --bootstrap-server kafka:19092 \
 --group s9981_writers \
 --describe
```

The corrected command ran successfully because it addressed the script at its actual location in the container. It provided objective offset and lag evidence for the saved HW8 records. This was a CLI invocation correction; it did not require changing the Kafka application logic.

Evidence: `reports/hw08/RUN_LOG.txt`, `reports/hw08/raw/part3_consumer_offsets_final.txt`, and the Part 1–3 experiment records in `reports/hw08/raw/`. AI guidance was checked against observed program output; no unexecuted command was represented as a successful test.
```